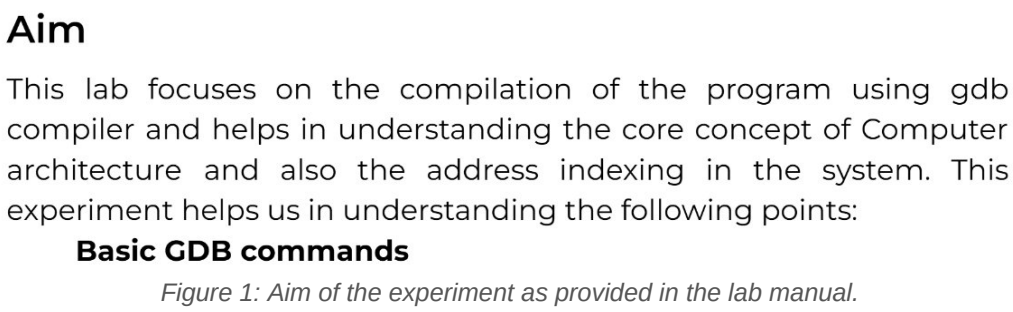
Experiment 4

1. Aim



Aim

This lab focuses on the compilation of the program using gdb compiler and helps in understanding the core concept of Computer architecture and also the address indexing in the system. This experiment helps us in understanding the following points:

Basic GDB commands

Figure 1: Aim of the experiment as provided in the lab manual.

2. Code

The following C program is used for this experiment. It contains two functions — **sum()** and **subtract()** — which are called from **main()**. This structure makes it easy to demonstrate stepping into functions during GDB debugging.

```
#include <stdio.h>

int sum(int a, int b) {
int result = a + b;
return result;
}

int subtract(int a, int b) {
int result = a - b;
return result;
}

int main() {
int x = 10, y = 4;
int s = sum(x, y);
int d = subtract(x, y);
printf("Sum : %d\n", s);
printf("Subtract : %d\n", d);
return 0;
}
```

[Insert Screenshot: C code open in VS Code / text editor]

3. Procedure

1. Open a terminal and navigate to the folder containing the C program.
2. Compile the program with debug symbols using: **gcc -g program.c -o program**
3. Launch GDB with the compiled binary: **gdb ./program**

4. Apply breakpoints, run the program, and step through using GDB commands as shown in Section 4.
5. Observe variable values and execution flow at each step.

4. GDB Commands and Output

Step 1 — Compile with Debug Symbols

The **-g** flag tells GCC to include debug information in the binary, which GDB needs to map machine instructions back to source lines.

```
$ gcc -g program.c -o program
$ ls
program program.c
```

[Insert Screenshot: Terminal — compilation command]

Step 2 — Launch GDB

Start GDB by passing the compiled binary. GDB prints version info and the **(gdb)** prompt appears, ready to accept commands.

```
$ gdb ./program
GNU gdb (Ubuntu 12.1) ...
Reading symbols from ./program...
(gdb)
```

[Insert Screenshot: Terminal — GDB launched]

Step 3 — list

The **list** command displays the source code inside GDB so you can identify the line numbers needed for setting breakpoints.

```
(gdb) list
1 #include <stdio.h>
2
3 int sum(int a, int b) {
4 int result = a + b;
5 return result;
6 }
7
8 int subtract(int a, int b) {
9 int result = a - b;
10 return result;
11 }
12
13 int main() {
14 int x = 10, y = 4;
15 int s = sum(x, y);
```

[Insert Screenshot: GDB Output — list command]

Step 4 — break (3 Breakpoints)

The **break** command pauses execution at a specified line. We set three breakpoints: at **main()** entry (line 13), at the **sum()** function call (line 15), and inside **sum()** where the addition happens (line 4).

```
(gdb) break 13
Breakpoint 1 at 0x1169: file program.c, line 13.

(gdb) break 15
Breakpoint 2 at 0x118a: file program.c, line 15.

(gdb) break 4
Breakpoint 3 at 0x1149: file program.c, line 4.
```

[Insert Screenshot: GDB Output — break commands]

Step 5 — run

The **run** command starts program execution from the beginning. GDB stops at the first breakpoint (line 13 — start of main).

```
(gdb) run
Starting program: /home/user/program

Breakpoint 1, main () at program.c:13
13 int x = 10, y = 4;
```

[Insert Screenshot: GDB Output — run command, stopped at BP1]

Step 6 — next (used twice)

The **next** command executes the current line and moves to the next line *without* stepping into function calls. It treats a function call as a single step.

```
(gdb) next
14 int x = 10, y = 4;

(gdb) next
Breakpoint 2, main () at program.c:15
15 int s = sum(x, y);
```

[Insert Screenshot: GDB Output — next command (x2)]

Step 7 — step (used twice, stepping into sum())

The **step** command also advances one line, but *steps into* function calls. Here it enters the **sum()** function and stops at line 4 (Breakpoint 3), allowing inspection of the function's internals.

```
(gdb) step
Breakpoint 3, sum (a=10, b=4) at program.c:4
4 int result = a + b;

(gdb) step
5 return result;
```

[Insert Screenshot: GDB Output — step into sum()]

Step 8 — print (inspect variables)

The **print** command evaluates and displays the value of a variable at the current point in execution. We print **a**, **b**, and **result** inside **sum()** to confirm the computation.

```
(gdb) print a
$1 = 10

(gdb) print b
$2 = 4

(gdb) print result
$3 = 14
```

[Insert Screenshot: GDB Output — print variables inside sum()]

Step 9 — continue

The **continue** command resumes execution until the next breakpoint or the end of the program. Since there are no more breakpoints after **sum()**, the program runs to completion and prints the final output.

```
(gdb) continue
Continuing.
Sum : 14
Subtract : 6
[Inferior 1 (process 12345) exited normally]
```

[Insert Screenshot: GDB Output — continue, program completes]

5. Result

The C program was successfully compiled with debug symbols using **gcc -g** and debugged using GDB. All seven GDB commands were demonstrated: **list** displayed source lines; **break** set three breakpoints at strategic locations; **run** started execution and halted at the first breakpoint; **next** stepped over lines in main(); **step** stepped into the **sum()** function; **print** confirmed variable values (a=10, b=4, result=14); and **continue** completed execution, printing the correct outputs: **Sum = 14** and **Subtract = 6**.